

Your University

Your Faculty

Your Study Program

OSS Risk Radar: Risk Assessment for Open Source Dependencies

Bachelor Thesis

Author:	Your Name
Matriculation number:	000000
Supervisor:	Prof. Dr. Supervisor Name
Second supervisor:	Second Supervisor Name
Submission date:	June 3, 2026

Abstract

This thesis investigates how open source dependency risk can be assessed by combining repository metadata, security signals, maintenance activity, and explainable scoring. Replace this placeholder with the final abstract once the research question, method, and results are fixed.

Declaration of Authorship

I declare that I wrote this thesis independently and used only the sources and tools listed in the bibliography and acknowledgements.

Place, date

Signature

Contents

Abstract	II
Declaration of Authorship	III
List of Figures	VI
List of Tables	VII
List of Abbreviations	VIII
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Research Question and Objectives	2
1.3 Scope and Assumptions	3
1.4 Contributions	4
1.5 Structure of the Thesis	4
2 Background	5
2.1 Software Supply-Chain Security	5
2.2 Open-Source Sustainability and Project Health	6
2.3 Metric Pitfalls and Limitations	7
2.4 Machine Learning Basics for Tabular Risk Scoring	8
2.5 Summary	9
3 Methodology	10
3.1 Research Design	10
3.2 Definitions and Prediction Target	11
3.3 Data Sources	11
3.4 Dataset Construction Workflow	12
3.5 Observation Window and Historical Coverage	13
3.6 Feature Engineering	14
3.7 Leakage Prevention	15
3.8 Model Training	15

3.9	Evaluation Plan	16
3.10	Reproducibility	17
3.11	Methodological Limitations	18
3.12	Summary	18
4	Implementation	19
4.1	Example Listing	19
5	Evaluation	20
6	Conclusion	21
	Bibliography	22
A	Additional Material	25

List of Figures

List of Tables

3.1	Historical dataset construction workflow	12
3.2	Implemented historical feature groups	14
3.3	Dataset and training run documentation template	17

List of Abbreviations

API	Application Programming Interface
CVSS	Common Vulnerability Scoring System
ML	Machine Learning
OSS	Open Source Software

1 Introduction

1.1 Motivation and Problem Statement

Modern software systems are not developed in isolation, but are commonly assembled from first-party code, third-party components, and open-source software dependencies. In this thesis, open-source software is understood as software distributed under a license that complies with the Open Source Definition, which includes rights such as use, modification, and redistribution [1]. From a secure software development perspective, such dependencies are relevant because the NIST Secure Software Development Framework explicitly includes the acquisition and maintenance of well-secured third-party and open-source components as part of secure development practice [2]. Similarly, the SLSA framework addresses software supply-chain integrity by focusing on artifacts, build processes, provenance, and related security controls [3].

The practical relevance of dependency management becomes visible in both empirical studies and real-world incidents. Miller et al. studied open-source dependency abandonment in the npm ecosystem and found that abandonment is common among widely used packages, while downstream projects often remain exposed to abandoned dependencies [4]. Their work also emphasizes that indirect dependencies can create wider consequences, because developers may have limited ability to react when an abandoned package appears only transitively in their dependency graph [4]. In an earlier interview study, Miller et al. found that developers who experienced dependency abandonment often lacked clear strategies and felt that they were improvising when responding to such situations [5].

Dependency-related risk is not limited to abandonment alone. Empirical work on vulnerable dependencies shows that known dependency vulnerabilities can persist for substantial periods. Prana et al. analyzed open-source dependencies in 450 Java, Python, and Ruby projects and found that many dependency vulnerabilities persisted throughout the observation period, while fixed vulnerabilities took several months to resolve on average [6]. Such findings indicate that delayed maintenance and delayed updates can create long-lived exposure in downstream software.

Real-world incidents further illustrate the potential impact of dependency problems. In 2016, the unpublishing of the small npm package `left-pad` caused a disruption for many projects that depended on it directly or indirectly, leading npm to manually restore the package and later change its unpublishing policy [7]. In 2017, Equifax disclosed that attackers exploited an Apache Struts vulnerability in its online dispute portal; the Federal Trade Commission later announced a settlement of at least \$575 million and potentially up to \$700 million related to the breach [8], [9]. More recently, the XZ Utils backdoor showed that open-source project maintenance and maintainer trust can become part of the attack surface: OpenSSF and the OpenJS Foundation described the case as a warning sign for social-engineering takeover attempts against open-source projects [10].

These examples do not imply that every inactive repository is insecure, abandoned, or harmful. A project may be stable, temporarily paused, or maintained through channels that are not fully visible in public repository data. However, they show that dependency health, maintainability, and responsiveness are relevant for secure software supply chains. Therefore, this thesis treats repository inactivity not as a definitive indicator of project death, but as an operational proxy for elevated maintenance risk. The central question is whether publicly observable repository signals can help estimate this risk early enough to support dependency triage and monitoring.

1.2 Research Question and Objectives

The central research question of this thesis is:

To what extent can a machine-learning-based inactivity-risk scoring tool, using only publicly observable repository health indicators and security-practice signals, predict 12-month OSS dependency inactivity and support risk-based dependency triage in secure dependency management?

To answer this research question, the thesis pursues the following objectives:

1. Define an operational inactivity label for open-source repositories within a 12-month prediction horizon.
2. Construct a reproducible dataset from publicly observable repository metadata and dependency-related sources.
3. Engineer activity, maintenance, and selected security-practice features based on repository and project health indicators.

4. Train and compare simple baselines and machine-learning models for inactivity-risk prediction.
5. Evaluate the models using discrimination and calibration metrics that are suitable for risk scoring.
6. Demonstrate how the trained model can be applied to a concrete dependency set to support risk-based triage.

The research follows a Design Science Research-oriented approach because the thesis develops and evaluates an artifact: an inactivity-risk scoring workflow for open-source dependencies [11], [12]. The empirical part evaluates whether the artifact produces useful predictive signals, while the practical demonstrator shows how the scoring workflow can be integrated into dependency monitoring.

1.3 Scope and Assumptions

The scope of this thesis is limited to open-source software dependencies whose source repositories and package metadata are publicly observable. The empirical dataset focuses on repositories that can be associated with an open-source license. Where GitHub license metadata is used, matched license keys and names are treated according to GitHub’s SPDX-based license information [13]. The thesis does not investigate private repositories, closed-source vendor components, or dependencies whose source repository cannot be mapped with sufficient confidence.

The prediction target is repository inactivity within the interval $[t, t + 12 \text{ months}]$. Inactivity is operationalized primarily through the absence of commits and releases or tags during the prediction horizon. This definition follows the idea that revision activity can be used as an observable signal for survival-style analysis of open-source projects [14]. Issue and pull-request activity are treated as predictor variables instead of label components, because they can indicate maintenance interaction without necessarily changing the released software artifact. Responsiveness-related features are interpreted cautiously, since bot-generated responses can distort measurements such as time to first response in pull-request workflows [15].

The thesis assumes that publicly observable repository signals can provide useful but incomplete evidence about future inactivity. It does not assume that these signals fully explain project sustainability. This distinction is important because previous research shows that cross-project health indicators can have different meanings depending on project context [16]. As a result, the model output is interpreted as a triage aid rather

than as an automated decision system.

1.4 Contributions

This thesis makes the following contributions:

1. It defines a practical 12-month inactivity-risk label for open-source dependency repositories.
2. It proposes a reproducible data collection and feature engineering workflow based on public repository, package, and dependency metadata.
3. It evaluates whether repository health indicators and security-practice signals can support machine-learning-based inactivity prediction.
4. It compares predictive models against simple baselines and assesses both discrimination and probability calibration.
5. It demonstrates how an inactivity-risk score can be applied to a dependency set in order to produce a ranked triage view for secure dependency management.

These contributions are intended to support software teams that need to prioritize dependency review and monitoring effort. The goal is not to replace expert judgement, but to provide a measurable and reproducible signal that can help identify dependencies that may deserve closer attention.

1.5 Structure of the Thesis

The remainder of this thesis is structured as follows. Chapter 2 introduces the theoretical background and related work, including software supply-chain security, open-source sustainability, project health metrics, and machine-learning basics for tabular risk scoring. Chapter 3 presents the research design, inactivity label, dataset construction, feature engineering approach, models, baselines, and evaluation plan. Chapter 4 describes the technical implementation of the data pipeline and scoring workflow. Chapter 5 reports and discusses the empirical results, including predictive performance, calibration, feature importance, and error analysis. Chapter 6 concludes the thesis and outlines limitations and opportunities for future work.

2 Background

This chapter introduces the conceptual foundations required for the thesis. It first explains the software supply-chain security context in which open-source dependencies are used and assessed. It then discusses open-source software sustainability and repository health metrics. Afterwards, it outlines important limitations of metric-based project assessment. Finally, it introduces the machine-learning concepts that are relevant for tabular inactivity-risk scoring.

2.1 Software Supply-Chain Security

Modern software systems are often composed of internally developed code and externally maintained components. In secure software development, these externally maintained components include commercial, third-party, and open-source software dependencies. The NIST Secure Software Development Framework explicitly treats the acquisition and maintenance of well-secured software components as part of secure development practice [2]. Therefore, dependency management is not only a build or maintenance concern, but also part of software supply-chain security.

Software supply-chain security focuses on reducing risks that arise when software artifacts, build processes, dependencies, and distribution channels are not sufficiently controlled or verifiable. The SLSA framework addresses this problem by defining security-oriented requirements for software artifacts and build processes, including provenance and integrity-related controls [3]. In the context of this thesis, these frameworks motivate why dependencies should be monitored beyond simple version updates.

Open-source dependencies are especially relevant because they are reused across many downstream projects. Open-source software is defined by licensing conditions that allow use, modification, and redistribution under criteria specified by the Open Source Initiative [1]. These freedoms enable broad reuse, but they also mean that downstream users often depend on external maintainers and public project infrastructure. As a

result, the maintenance activity of an upstream repository can become relevant for downstream risk assessment.

For this thesis, software supply-chain security provides the practical context for inactivity-risk scoring. The goal is not to determine whether a dependency is secure or insecure in general. Instead, the goal is to estimate whether publicly observable repository signals indicate an elevated risk of future inactivity. Such a score can support dependency monitoring, replacement planning, or prioritization of manual review.

2.2 Open-Source Sustainability and Project Health

Open-source sustainability describes whether a project can continue to provide value and maintain its software over time. Project health is commonly assessed through observable indicators such as activity, responsiveness, contributor participation, and release behavior. The CHAOSS project provides standardized metrics and metrics models for understanding open-source community and project health [17]. Its Starter Project Health Metrics Model is intended as an entry point for measuring selected aspects of project health in a structured way [17].

Repository activity is one category of observable project health. Examples include commits, releases, tags, issues, pull requests, and contributor activity. These signals are useful because they can often be collected from public repository platforms and package metadata sources. Prior research has used revision activity and project-level attributes to study open-source project survival and inactivity-related outcomes [14]. In this thesis, such activity signals form the basis for predicting whether a dependency repository becomes inactive within a 12-month horizon.

Security-practice indicators form a second category of observable signals. OpenSSF Scorecard is an automated tool that assesses open-source projects using security-related checks [18]. These checks do not directly measure future maintenance activity, but they may provide additional information about project practices, repository configuration, and security posture. Therefore, selected Scorecard checks can be considered as candidate features for an inactivity-risk model.

However, project health is broader than any single metric. Goggins et al. argue that making open-source project health transparent requires structured measurement and interpretation rather than isolated indicators [19]. This is important for the thesis because inactivity-risk scoring should not be interpreted as a complete judgement of project sustainability. Instead, the score is treated as a limited operational signal for

dependency triage.

2.3 Metric Pitfalls and Limitations

Although repository metrics are useful, they can be misleading when interpreted without context. Yehudi et al. show that individual context-free online community health indicators are insufficient for identifying open-source software sustainability across heterogeneous projects [16]. This means that a metric value can have different meanings depending on the project type, maturity, governance model, and maintenance style.

One example is commit activity. A low number of recent commits may indicate reduced maintenance, but it may also indicate that a project is stable and does not require frequent changes. Similarly, a high number of issues may indicate an active user community, but it may also indicate unresolved maintenance problems. Therefore, this thesis does not treat single metrics as direct proof of project health or project failure. Instead, multiple signals are combined and evaluated empirically.

Responsiveness metrics also require caution. For example, time-to-first-response can be affected by automated bot responses in pull-request workflows. Hasan et al. show that bot-generated first responses can influence the interpretation of responsiveness measurements in GitHub pull requests [15]. For this reason, responsiveness-related indicators should be interpreted carefully and, where possible, separated into human and automated activity.

A further limitation is that public repository data may be incomplete. Development may move to another platform, releases may be published outside the observed repository, or maintainers may communicate through channels that are not captured by GitHub metadata. Package-to-repository mappings may also be ambiguous or missing. Therefore, the empirical results of this thesis must be interpreted as evidence based on publicly observable data, not as a complete representation of all project activity.

These limitations influence the design of the inactivity label. Inactivity is not defined as project death or abandonment. Instead, it is operationalized as the absence of selected observable activity signals within a defined prediction horizon. This makes the label measurable and reproducible, while still acknowledging that it is only a proxy for elevated maintenance risk.

2.4 Machine Learning Basics for Tabular Risk Scoring

The prediction task in this thesis is a binary classification problem. At a given observation time t , a repository is represented by a set of features derived from public metadata and activity signals. The target variable indicates whether the repository becomes inactive in the interval $[t, t + 12 \text{ months}]$. The model therefore estimates the probability that a dependency repository will become inactive within the prediction horizon.

A simple baseline is required to determine whether machine learning provides value beyond straightforward heuristics. For this thesis, a recency-based baseline can use the time since the last observed commit or release as a simple inactivity-risk indicator. Such a baseline is important because repository inactivity is closely related to recent activity by definition. A machine-learning model is only useful if it improves over this simpler reference point.

Logistic regression can be used as an interpretable probabilistic baseline for binary classification. Tree-based models, such as random forests and gradient boosting, can capture non-linear relationships and interactions between features [20], [21], [22]. These properties make them suitable candidates for tabular repository metadata, where activity, maintenance, and security-practice indicators may interact in non-linear ways.

Model evaluation should consider both discrimination and calibration. Discrimination describes how well a model separates inactive from active repositories. Common metrics include the area under the receiver operating characteristic curve and the area under the precision-recall curve. Precision-recall evaluation is especially relevant when the positive class is rare or imbalanced [23]. Calibration describes whether predicted probabilities correspond to observed outcome frequencies. This is important because a risk score should be interpretable as a probability-like triage signal rather than only as a ranking [24].

To avoid overly optimistic evaluation, the dataset should be split in a time-aware way. A random split could allow information from later periods to influence model development for earlier periods, which would not reflect the real prediction setting. A time-aware split better matches the intended use case: using historical repository information to predict future inactivity.

2.5 Summary

This chapter established the background for the thesis. Software supply-chain security motivates the need to monitor open-source dependencies as externally maintained components. Open-source project health metrics provide observable signals that can be used for inactivity-risk prediction. However, such metrics are context-dependent and can be distorted by project-specific practices, automation, or incomplete data. Machine-learning-based risk scoring can combine multiple signals, but it must be evaluated against simple baselines and assessed using both discrimination and calibration metrics.

The following chapter discusses related work on open-source inactivity, repository health measurement, and security-practice indicators in more detail.

3 Methodology

This chapter describes the methodological design used to answer the research question. The thesis follows a Design Science Research-oriented approach, because the work develops and evaluates an artifact for a practical software engineering problem: an inactivity-risk scoring workflow for open-source dependencies [11], [12]. The artifact is implemented in OSS Risk Radar and combines dependency identification, repository mapping, historical data construction, feature engineering, supervised model training, and risk-oriented output for dependency triage.

The empirical part of the method is based on historical supervised learning. For each repository, the dataset builder creates observation snapshots at defined points in time. Each snapshot contains features derived only from information available at or before the observation date t . The label is derived from the following 12-month period $(t, t + 12 \text{ months}]$. This design reflects the intended use case of predicting whether a repository is likely to become inactive after the observation date, rather than explaining inactivity after it has already happened.

3.1 Research Design

The research design consists of three connected parts. First, a historical dataset is constructed from real open-source repositories and public event data. Second, a supervised classification model is trained to predict 12-month inactivity. Third, the trained model is evaluated and interpreted as a decision-support signal for dependency triage.

The artifact is not designed as a vulnerability scanner or as a definitive trust score. Instead, it provides a risk-oriented ranking based on observable maintenance and activity signals. This distinction is important because inactivity is only one aspect of dependency risk. A repository can be inactive without being insecure, and a repository can be active while still having security weaknesses. Therefore, the model output is interpreted as an operational triage signal, not as an automated replacement for expert judgement.

3.2 Definitions and Prediction Target

The central unit of analysis is a GitHub repository that can be associated with an open-source package or dependency candidate. A snapshot is defined by a repository, an optional package identity, and an observation date t . The prediction task is binary classification:

Given the observable repository and package state at time t , predict whether the repository becomes inactive within the following 12 months.

The implemented training label is named `label_inactive_12m`. Internally, the dataset builder first computes a maintenance variable named `maintained_12m`. The inactivity label is then derived as the inverse of this maintenance variable. If the future window is incomplete, the row remains unlabeled and is not used for supervised model training.

Formally, the future label window is defined as:

$$(t, t + 12 \text{ months}]$$

The current implementation treats a repository as maintained within the 12-month horizon if it is not archived or deleted by $t + 12$ months and if at least two future-window activity checks are fulfilled. These checks are:

1. human commits occur in at least three distinct months,
2. at least two contributors are active,
3. at least one release or package version publication occurs,
4. at least two pull requests are merged.

This definition deliberately avoids treating a single event as sufficient evidence of maintenance. At the same time, it remains operationally measurable from historical public data. The merged-pull-request condition is used as a practical proxy for broader maintainer interaction, because GH Archive data does not reliably distinguish all maintainer responses without additional issue-comment attribution logic.

3.3 Data Sources

The dataset construction uses public and reproducible data sources wherever possible. GH Archive is used as the primary source for historical GitHub event data, because

it provides public GitHub event streams as hourly archives [25]. The dataset builder parses these event files into normalized repository histories, including commits, issues, pull requests, releases, stars, and forks.

The package-to-repository mapping is based on explicit seed fields where available. If the seed does not already contain a repository identity, the builder can resolve repository mappings through deps.dev and package registry metadata. deps.dev provides open-source package and dependency metadata across supported ecosystems [26]. GitHub metadata is used only for stable repository fields such as repository creation time, default branch, and fork state. Current GitHub totals for historical activity are avoided because they would leak present-day information into older observation snapshots.

The repository also contains a deterministic real-project seed file:

```
scripts/ml/real-project-foundation-seed.csv
```

This seed contains active, dormant, and retired project candidates. These categories are used only for candidate selection and are not used as labels. The final labels are derived exclusively from the historical future window.

3.4 Dataset Construction Workflow

The historical dataset builder creates an inspectable dataset through a sequence of intermediate steps. The implemented workflow is summarized in Table 3.1.

Table 3.1: Historical dataset construction workflow

Step	Description
1	Load package or repository candidates from a seed file.
2	Stratify and sample candidates by ecosystem and popularity information where available.
3	Resolve repository mappings from explicit seed fields, deps.dev, or package registry metadata.
4	Enrich stable repository metadata through GitHub.
5	Parse GH Archive event files into normalized repository histories.
6	Build observation snapshots for the configured observation period.
7	Compute feature rows using only data available at or before the observation date t .
8	Compute 12-month labels using only the future window $(t, t + 12 \text{ months}]$.
9	Export the final training snapshot file for the existing model trainer.

The builder writes intermediate files under the configured output directory. These files make the dataset construction process auditable and allow the generated dataset to be inspected before training. The intermediate outputs include:

- `repositories.jsonl`
- `packages.jsonl`
- `package_repository_links.jsonl`
- `observation_snapshots.jsonl`
- `snapshot_features.jsonl`
- `snapshot_labels.jsonl`
- `training_snapshots.json`

The final export follows the existing training snapshot contract used by OSS Risk Radar. This allows the historical dataset builder to feed the current training pipeline without requiring a separate model-training interface.

3.5 Observation Window and Historical Coverage

The default observation period used by the project is:

```
observation_start = 2023-01-01
observation_end    = 2024-01-01
```

The label horizon is 12 months. The feature history can look back up to 730 days. Therefore, a complete GH Archive source range for the default setup should cover approximately:

```
2021-01-01 through 2025-01-01
```

The first part of this range supports feature construction before the earliest observation date. The final part supports label construction after the latest observation date. If the GH Archive coverage does not include the full future label window for a snapshot, the builder marks the label as incomplete and does not use that row as a labeled training example.

For local development, a smaller filtered GH Archive subset can be used to test the pipeline. However, sparse random hours are not sufficient for thesis-grade labels, because the absence of events would no longer be meaningful. For supervised training claims, all relevant hours in the selected historical period should be processed, at least for the repositories included in the seed set.

3.6 Feature Engineering

The feature engineering step converts repository and package history before t into numeric variables. The implemented feature groups are shown in Table 3.2.

Table 3.2: Implemented historical feature groups

Feature group	Example features
Commit activity	Commits in the last 30, 90, and 365 days; active commit months; days since last commit.
Contributor activity	Contributors in the last 90 and 365 days; new contributors; contributor concentration.
Issue activity	Opened and closed issues; issue closure ratio; issue backlog growth; stale open issues.
Pull request activity	Opened, merged, and closed-unmerged pull requests; pull request merge ratio; stale open pull requests.
Release activity	Releases in the last 365 days; days since last release; versions published in the last 365 days.
Age and popularity proxies	Package age; repository age; stars and forks observed in the available GH Archive history.
Risk proxy features	Activity drop; contributor drop; release gap risk; concentration risk score.
Repository state	Archive status at observation time.

Some variables are practical proxies rather than complete measurements. For example, stars and forks are reconstructed from the provided GH Archive window and therefore represent lower-bound proxies if the archive window starts after the repository was created. Similarly, issue backlog growth and pull-request response features are reconstructed from available event data and should not be interpreted as canonical CHAOSS metrics. This limitation is documented because repository health metrics can be context-dependent and should not be interpreted as universal sustainability indicators [16].

3.7 Leakage Prevention

Temporal leakage is a central threat in historical prediction tasks. The dataset builder applies the following rules to reduce leakage:

1. Observation-time features are derived only from timestamps $\leq t$.
2. Labels are derived only from timestamps in $(t, t + 12 \text{ months}]$.
3. Rows with incomplete future-window coverage remain unlabeled.
4. Live GitHub enrichment is limited to stable metadata such as creation time, default branch, and fork state.
5. Historical activity totals are not taken from current GitHub API totals.

These rules are necessary because the model is intended to simulate a real prediction setting. At observation time t , future activity is unknown. Therefore, any information from after t must be excluded from feature construction and used only for label derivation.

3.8 Model Training

The current runtime training pipeline supports a logistic-regression baseline model. Logistic regression is suitable as an initial supervised baseline because it produces probability-like outputs and is more interpretable than many complex non-linear models [20]. More complex tabular models, such as gradient boosting or ensemble methods, are treated as possible extensions unless they are implemented and evaluated before the final thesis submission.

Training uses a time-aware split with the default ratio:

75% training, 15% validation, 10% test

Rows are ordered by observation date before splitting. The model is fitted on the earliest training slice. The validation slice is used for calibration and threshold selection. The final test slice is held out for reporting evaluation results. This design avoids a random split that could mix earlier and later observation periods in a way that does not match the intended forecasting use case.

The implemented pipeline fits the logistic-regression model, predicts validation proba-

bilities, fits a histogram-based calibrator on validation predictions, selects a decision threshold from the validation data, and then evaluates calibrated predictions on the test data. Calibration is relevant because the output should support risk triage as a probability-like score, not only as a binary classification result [24].

3.9 Evaluation Plan

The evaluation focuses on whether the model provides a useful inactivity-risk signal for dependency triage. The implemented training pipeline reports:

- sample count,
- positive label rate,
- accuracy,
- precision,
- recall,
- F1 score,
- Brier score,
- log loss,
- ROC-AUC,
- model quality score.

ROC-AUC measures the model’s ability to rank inactive repositories above maintained repositories. Precision, recall, and F1 score evaluate the binary decisions after threshold selection. The Brier score evaluates the quality of predicted probabilities and is therefore relevant for calibrated risk scoring [24]. If the inactive class is strongly imbalanced, precision-recall analysis should additionally be reported, because precision-recall curves can be more informative than ROC curves for imbalanced binary classification problems [23].

The model is compared against simple baseline reasoning during interpretation. In particular, features such as days since last commit or days since last release are expected to be strong signals because the label itself is related to future activity. The machine-learning model is useful only if it provides value beyond such simple recency-based indicators.

3.10 Reproducibility

The dataset builder and training pipeline are executed through npm scripts from the repository root. For the deterministic real-project seed, the dataset can be built with:

```
npm run ml:dataset:real-projects -- '
  --gharchive-source .\tmp\gharchive '
  --output-dir .\tmp\training\oss-maintenance '
  --training-output-path .\tmp\training\snapshots.json
```

The combined build-and-train workflow can be executed with:

```
npm run ml:bootstrap:real-projects -- '
  --gharchive-source .\tmp\gharchive '
  --output-dir .\tmp\training\oss-maintenance '
  --training-output-path .\tmp\training\snapshots.json
```

For the final thesis evaluation, the executed command, GH Archive coverage period, seed file version, generated intermediate files, number of repositories, number of snapshots, number of labeled rows, inactive label rate, and model artifact timestamp should be documented.

Table 3.3: Dataset and training run documentation template

Item	Value
Seed file	scripts/ml/real-project-foundation-seed.csv
GH Archive source range	TODO: add final range
Observation start	TODO: add final value
Observation end	TODO: add final value
Label horizon	12 months
Feature lookback	TODO: add final value
Number of repositories	TODO: add after final run
Number of snapshots	TODO: add after final run
Number of labeled snapshots	TODO: add after final run
Inactive label rate	TODO: add after final run
Training command	TODO: add final command
Model artifact	TODO: add path or timestamp

3.11 Methodological Limitations

The method has several limitations. First, it depends on the completeness of the provided GH Archive files. If relevant event hours are missing, repository histories may be incomplete. Second, public GitHub activity does not capture all possible maintenance activity. Some projects may move development to other platforms, publish releases outside GitHub, or maintain stable software with little visible activity. Third, package-to-repository mappings can be ambiguous, especially when packages move repositories or when metadata is incomplete. Fourth, the inactivity label is a proxy for elevated maintenance risk and not a definitive statement that a project is abandoned or insecure.

These limitations are addressed through conservative interpretation. The model output is not treated as proof that a dependency is unsafe. Instead, it is used as a prioritization signal that can support manual dependency review and monitoring.

3.12 Summary

This chapter described the methodological design of the thesis. The approach constructs a historical supervised dataset from real repositories, derives observation-time features without future leakage, labels snapshots using a 12-month future window, and trains a logistic-regression baseline with time-aware validation and testing. The next chapter describes the implementation of OSS Risk Radar and explains how the dataset builder, scoring service, backend, and frontend are integrated into the overall artifact.

4 Implementation

Document the implemented system architecture, backend services, frontend views, scoring components, and data flow.

4.1 Example Listing

Listing 4.1: Example npm command

```
1 npm run thesis:build
```

5 Evaluation

Describe how the approach is evaluated. This can include sample repositories, manual review, metric comparisons, error analysis, and limitations.

6 Conclusion

Summarize the findings, answer the research question, and outline future work.

Bibliography

- [1] Open Source Initiative, *The open source definition*, [Online]. Available: <https://opensource.org/osd>. Accessed: 2026-02.
- [2] M. Souppaya, K. Scarfone, and D. Dodson, “Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-218, Feb. 2022. DOI: 10.6028/NIST.SP.800-218
- [3] SLSA Community, *Slsa specification v1.2*, [Online]. Available: <https://slsa.dev/spec/v1.2/>, Nov. 2025.
- [4] C. Miller, M. Jahanshahi, A. Mockus, B. Vasilescu, and C. Kästner, “Understanding the response to open-source dependency abandonment in the npm ecosystem,” in *Proc. IEEE/ACM 47th International Conference on Software Engineering*, 2025, pp. 2355–2367. DOI: 10.1109/ICSE55347.2025.00004
- [5] C. Miller, C. Kästner, and B. Vasilescu, ““we feel like we’re winging it:” a study on navigating open-source dependency abandonment,” in *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1281–1293. DOI: 10.1145/3611643.3616293
- [6] G. A. A. Prana et al., “Out of sight, out of mind? how vulnerable dependencies affect open-source projects,” *Empirical Software Engineering*, vol. 26, no. 4, 2021. DOI: 10.1007/s10664-021-09959-3
- [7] npm, Inc., *Kik, left-pad, and npm*, [Online]. Available: <https://blog.npmjs.org/post/141577284765/left-pad-and-npm>, Mar. 2016.
- [8] Equifax, *Equifax releases details on cybersecurity incident*, [Online]. Available: <https://investor.equifax.com/news-events/press-releases/detail/237/equifax-releases-details-on-cybersecurity-incident>, Sep. 2017.
- [9] Federal Trade Commission, *Equifax to pay \$575 million as part of settlement with ftc, cfpb, and states related to 2017 data breach*, [Online]. Available: <https://www.ftc.gov/news-events/news/press-releases/2019/07/equifax-pay-575-million-part-settlement-ftc-cfpb-states-related-2017-data-breach>, Jul. 2019.

- [10] Open Source Security Foundation and OpenJS Foundation, *Open source security and openjs foundations issue alert for social engineering takeovers of open source projects*, [Online]. Available: <https://www.linuxfoundation.org/blog/openssf-openjs-alert-for-social-engineering-takeovers-of-open-source-projects>, Apr. 2024.
- [11] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [12] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. DOI: 10.2753/MIS0742-1222240302
- [13] GitHub, *Rest api endpoints for licenses*, [Online]. Available: <https://docs.github.com/en/rest/licenses>, Accessed: 2026-02.
- [14] D. Robinson, K. Enns, N. Koulekar, and M. Sihag, *Two approaches to survival analysis of open source python projects*, arXiv:2203.08320, 2022.
- [15] K. A. Hasan, M. Macedo, Y. Tian, B. Adams, and S. H. H. Ding, “Understanding the time to first response in github pull requests,” in *Proc. IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023.
- [16] Y. Yehudi, C. Goble, and C. Jay, *Individual context-free online community health indicators fail to identify open source software sustainability*, arXiv:2309.12120, 2023.
- [17] CHAOSS Community, *Metrics model: Starter project health*, [Online]. Available: <https://chaoss.community/kb/metrics-model-starter-project-health/>, Accessed: 2026-02.
- [18] N. Zahan, P. Kanakiya, B. Hambleton, S. Shohanuzzaman, and L. Williams, *Openssf scorecard: On the path toward ecosystem-wide automated security metrics*, arXiv:2208.03412, 2022.
- [19] S. P. Goggins, M. Germonprez, and K. Lombard, “Making open source project health transparent,” *Computer*, vol. 54, no. 8, pp. 104–111, 2021. DOI: 10.1109/MC.2021.3084015
- [20] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed. Springer, 2021. DOI: 10.1007/978-1-0716-1418-1
- [21] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324

- [22] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. DOI: 10.1214/aos/1013203451
- [23] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432
- [24] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. 22nd International Conference on Machine Learning*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430
- [25] GH Archive, *Gh archive: Github archive data*, [Online]. Available: <https://www.gharchive.org/>, 2026.
- [26] deps.dev, *Deps.dev api*, [Online]. Available: <https://docs.deps.dev/api/v3/>, Accessed: 2026-02.

A Additional Material

Add extended tables, screenshots, raw evaluation notes, or other supporting material here.